

Parallel and Distributed Systems

Chapter 5: Thread-Based Parallelism

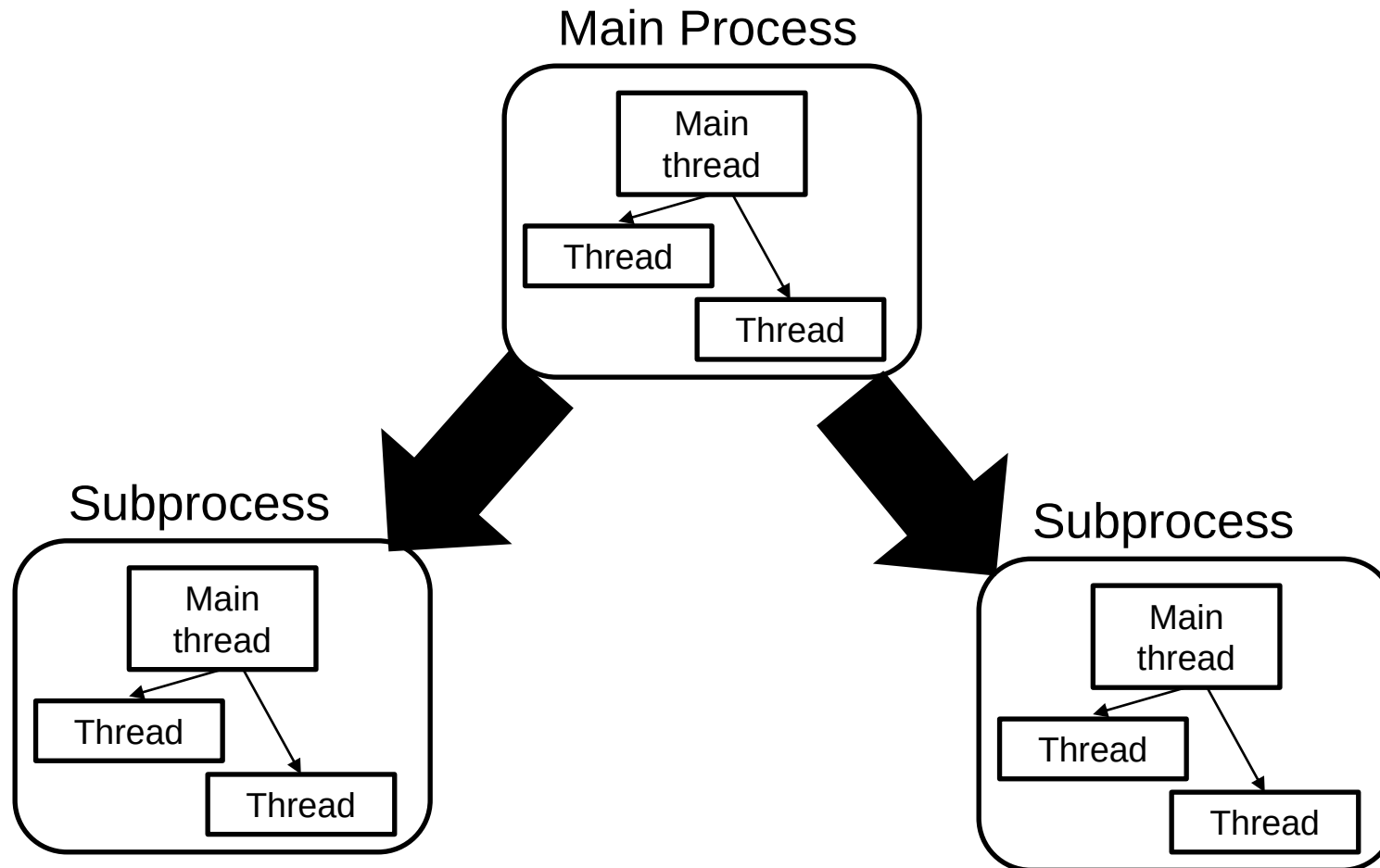
Prof. Dr. Martin Sträßer



Copyright & Acknowledgments

- All materials (incl. lecture and exercise slides, recordings, exercise tasks and sheets, programming tasks) are provided for personal use only. Reproducing, copying, uploading, sharing, or providing materials to third parties is not permitted

Recap: Structure of Concurrent Programs





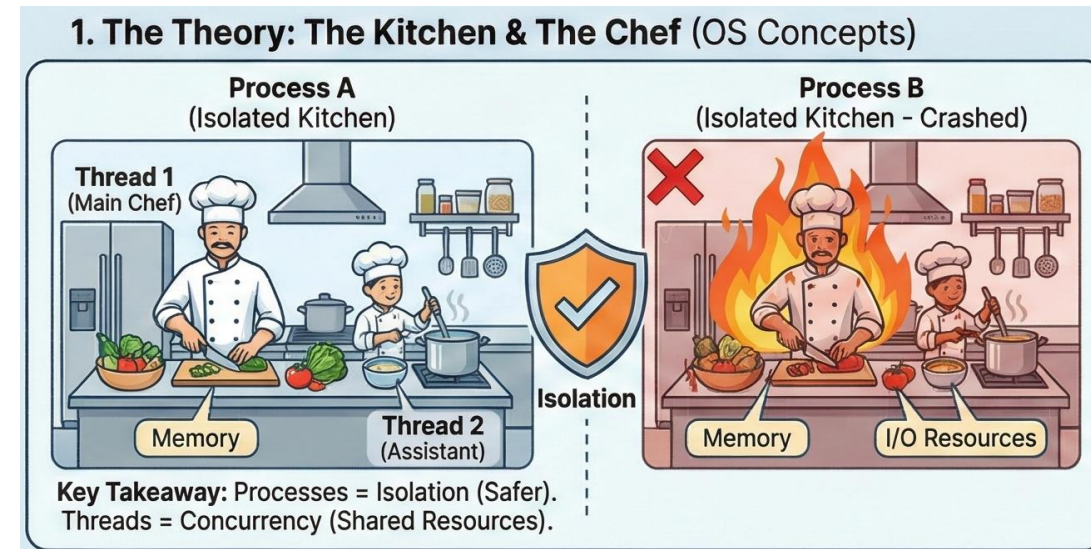
Overview

- Threads from an OS-level perspective
- Threads in Python

Threads from an OS-level perspective

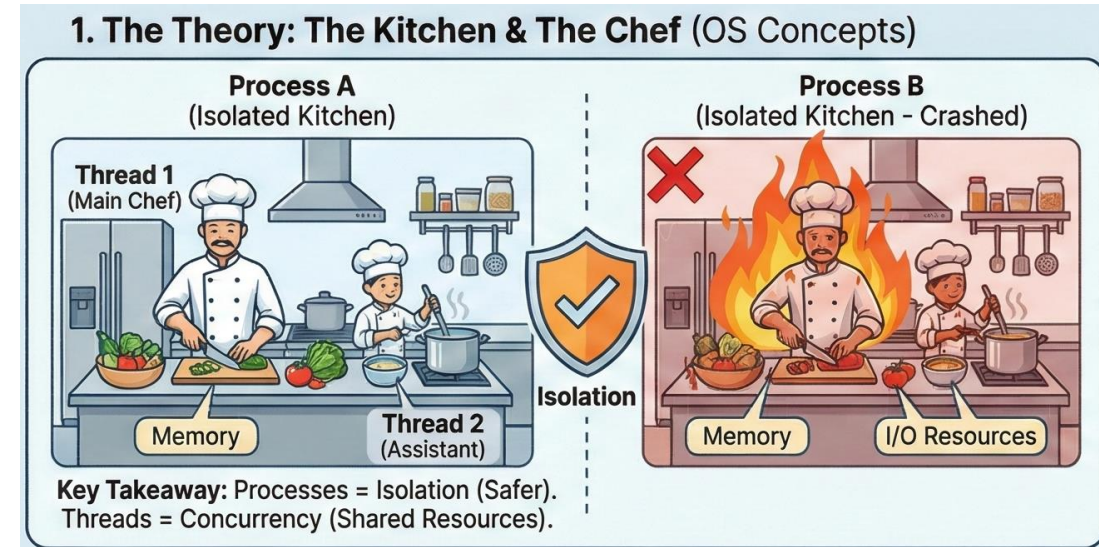
Threads

- A process can contain multiple threads
- A thread is an independent execution flow that can be executed in parallel and concurrently with other threads in the system
- Threads are the scheduling unit in operating systems (each core can run a single thread)
- Multiple threads in a process can share data and resources



Threads

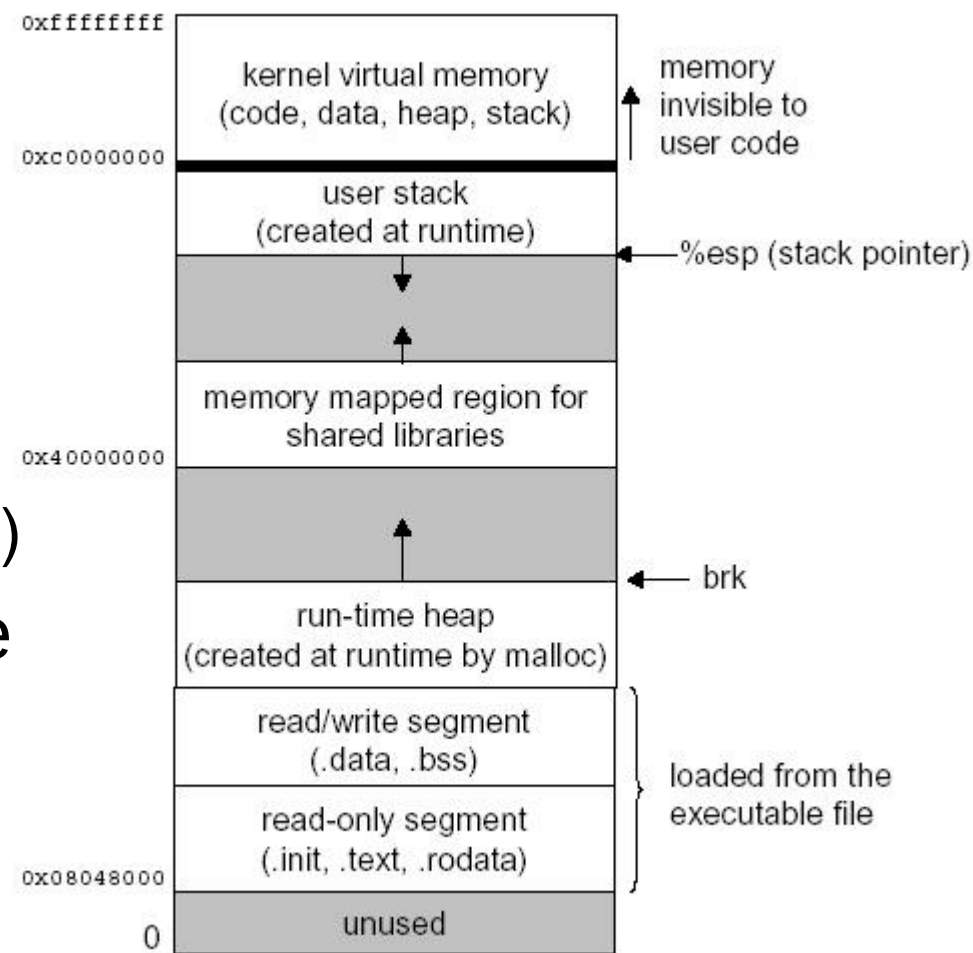
- The context switch between processes is much heavier than the switch between threads that belong to the same process
- Threads can use lightweight communication between each other as they can share variables





Recap: Process Components

- Virtual address space is the memory region where the process has access to, it contains:
 - Program code
 - Stack and heap
 - Environment/Global variables
 - Memory mappings (shared libraries, files, ...)
- There is also a kernel-level process state independent of the process memory, it contains:
 - Current working directory
 - Kernel metadata (PID, scheduling info, ...)



Thread Components

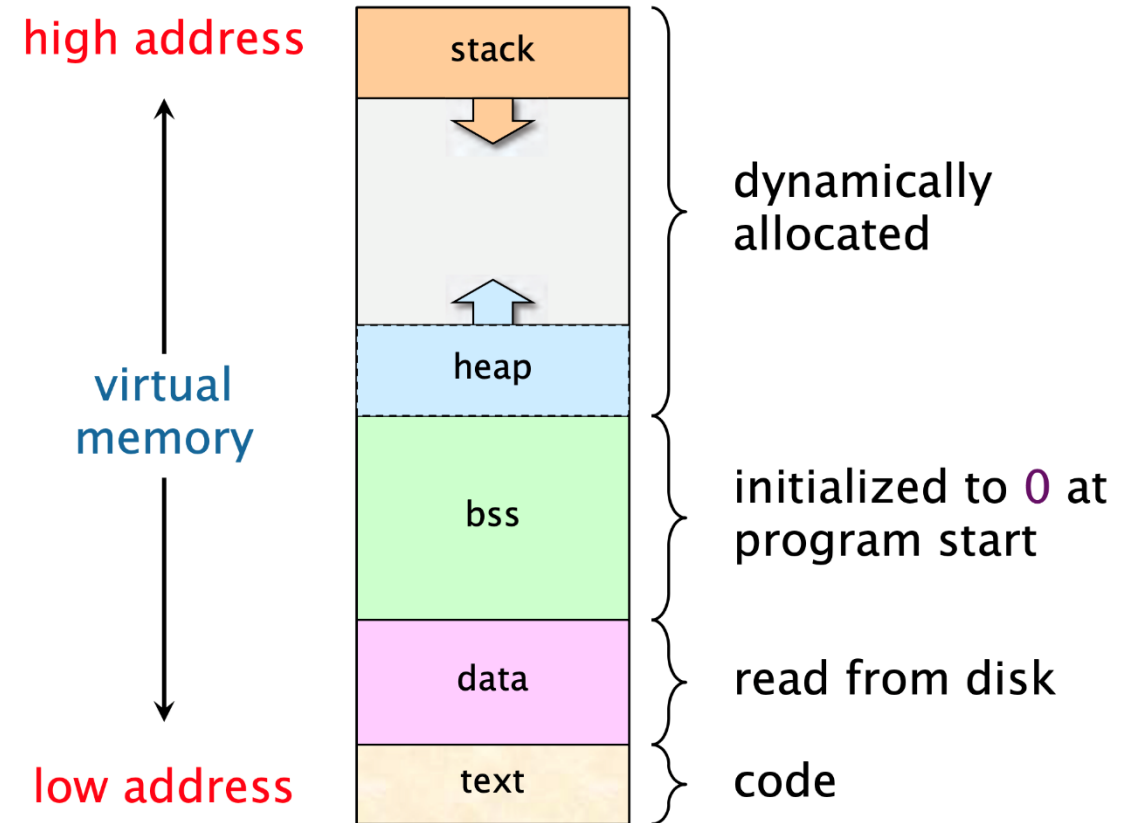
- Notably, threads within the same process share large parts of the process memory
- However, threads have:
 - Their own stack (for function calls and local variables)
 - Own signal handling (they share signal disposition, but signals are received by individual threads)
 - Own kernel entries (they share process information, but have also individual data)
 - Their own program counter and register states

Process Component	Shared by Threads
Program code	Yes
Stack	No
Heap	Yes
Environment variables	Yes
Open file descriptors	Yes
Signal handlers	Partially
Current working directory	Yes
Kernel metadata	Partially

This makes creating threads lightweight! There is not many data that has to be copied!

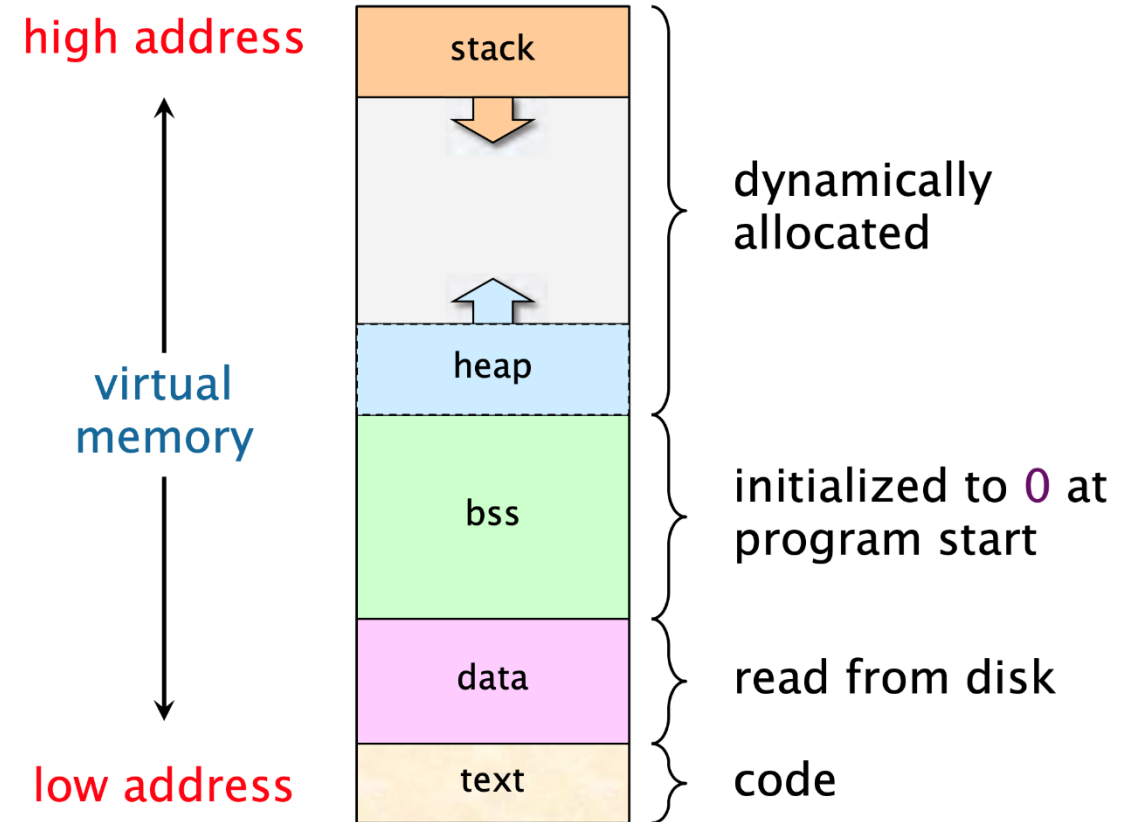
Question

- What is the difference between stack and heap memory?
- Which problem arises when threads have their own stack but not their own heap?



Thread-Local Storage

- What is difference between stack and heap memory?
- Which problem arises when threads have their own stack but not their own heap?
- Problem: How can a thread declare variables to be used by multiple functions, but not by other threads?
- Solution: **Thread-Local Storage**
- Similar to heap, but not shared with other threads





Threads vs. Processes

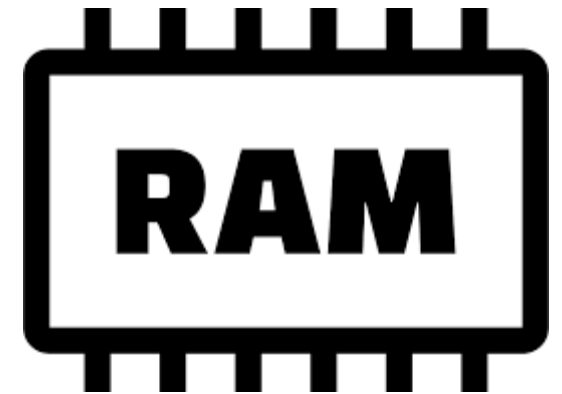
Feature	Threads	Processes
Memory sharing	Large parts of memory are shared	No shared memory by default
Creation	Fast	Slow
Context switches	Fast	Slow
Impact of failures	Depends	Failures in one process do not affect others
Security and isolation	Weak	Strong
Core challenge	Synchronized access to shared data	Inter-process communication

Thread Creation Steps

- Similar in all operating systems
- Allocation of the thread control block (TCB) in the kernel
 - Contains ID, CPU registers, stack pointer, program counter, state info, scheduling info, and more
- Allocation of stack memory and thread-local storage
- Setting up the execution context (thread memory)
 - Initialization of CPU registers, program counter, ...
 - References to shared memory with process
- Add thread to OS scheduler queue

Thread Termination

- Thread can terminate normally or with exception
- Forcefully termination is not as common as for processes (but possible, e.g., for daemon threads)
- State in TCB is updated
- Stack and thread-local storage is freed
- Notification of waiting threads and forward of return value
- Deletion from scheduler queue
- Deletion of TCB





When to Use Threads?

- For I/O-heavy tasks, like:
 - Network requests in web servers are handled in different threads
 - File downloads
 - Database calls
- When we need frequent access to shared resources, like global variables
- For relatively short-lived concurrent tasks (benefit: fast creation and termination)

Threads in Python



Python Threads

- Threads within a Python process share one instance of the Python interpreter that executes Python code
- Threads in Python do not behave as perfectly as threads in the operating system
- Working with threads in Python is like working with a single-core CPU
- Threads in Python cannot be executed simultaneously on two different cores
- Library for implementation: threading (not multithreading)



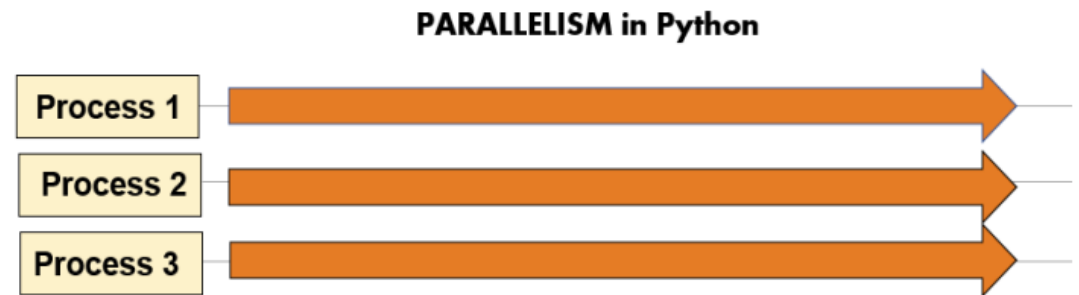
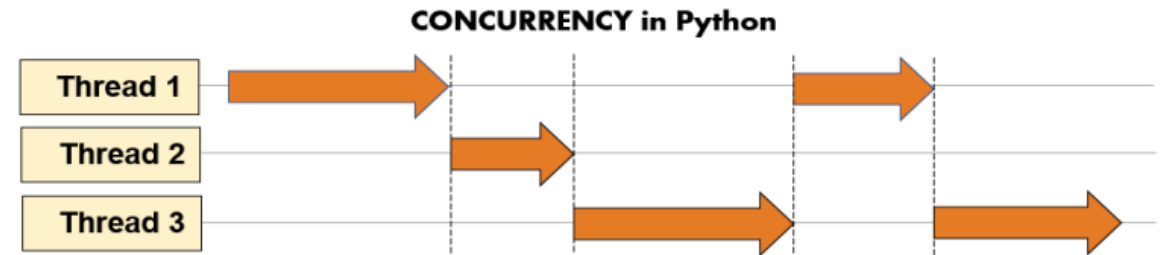
Global Interpreter Lock

- Reference implementation of interpreter is CPython
- CPython was not fully thread-safe: That is, the more threads tried to access a certain object in common, it often ran into a state of inconsistency
- Solution: **Global Interpreter Lock (GIL)** (only one thread can be executed at the same time)
- Consequence: concurrency, but no parallel execution
- **This is CPython-specific**
- GIL is optional in newest CPython version and will possibly be removed in future versions
- High-performance libraries like NumPy are implemented in C



Alternative Python Interpreter

- Why not change the Python interpreter?
 - There are other Python interpreters like PyPy, Jython, or IronPython
 - Jython (implemented in Java) and IronPython (C#) do not have a GIL and support multithreading
 - However, they have other problems like compatibility with essential libraries like NumPy
- Always remember: Python is in general a slow language!





When to Use Threads in Python

- Summary: Only one thread can run on the CPU at the same time
- This means that CPU-bound tasks (e.g., calculations) do not run faster when using threads
- When to use threads in Python?
 - Not all tasks are CPU-bound
 - Threads are used for I/O- or network-bound tasks
- Examples:
 - One thread waits for a file download to complete, main thread continues working
 - One thread waits for database write to complete
 - A web server (frontend) waits for data from the backend

Threads in Other Languages

- C/C++ give direct access to OS-level threads which leads to full control and also synchronization risks
- Java and C# provide high-level abstractions for multithreading and also many synchronization mechanisms
- Go supports concurrency by design with lightweight and safe goroutines
- Rust supports safe multithreading by design, potential race conditions lead to compiler errors
- JavaScript is single-threaded, relies heavily on asynchronous programming and events (later in this course), some multithreading mechanisms included in newest versions

Starting Threads

- Explain this example output

START PROGRAM

Start thread 1

Start thread 2

Start thread 3

Start thread 4

Start thread 5END PROGRAM

End thread 1

End thread 3

End thread 5

End thread 4

End thread 2

See code no. 1

```
import threading
import time

def function(i: int) -> None: 5 usages
    print(f"Start thread {i}")
    time.sleep(2)
    print(f"End thread {i}")

if __name__ == '__main__':
    print("START PROGRAM")
    t1 = threading.Thread(target=function, args=(1,))
    t2 = threading.Thread(target=function, args=(2,))
    t3 = threading.Thread(target=function, args=(3,))
    t4 = threading.Thread(target=function, args=(4,))
    t5 = threading.Thread(target=function, args=(5,))
    t1.start()
    t2.start()
    t3.start()
    t4.start()
    t5.start()
    print("END PROGRAM")
```



Join Threads

```
from threading import Thread
import time

def function(i: int) -> None: 2 usages
    print(f"Start thread {i}")
    time.sleep(2)
    print(f"End thread {i}")

if __name__ == '__main__':
    print("START PROGRAM")
    t1 = Thread(target=function, args=(1,))
    t2 = Thread(target=function, args=(2,))
    t1.start()
    t2.start()
    t1.join()
    t2.join()
    print("END PROGRAM")
```

See code no. 2

Pattern: Creation of Threads in a Loop

```
import threading
import time

def function(i: int) -> None: 1 usage
    print(f"Start thread {i}")
    time.sleep(2)
    print(f"End thread {i}")

if __name__ == '__main__':
    print("START PROGRAM")
    n_threads = 5
    threads = []
    for i in range(n_threads):
        t = threading.Thread(target=function, args=(i,))
        threads.append(t)
        t.start()
    for i in range(n_threads):
        threads[i].join()
    print("END PROGRAM")
```

See code no. 3



Execution Order of Threads

```
import threading
import time

sequence = []
countA = 6
timeA = 5
countB = 3
timeB = 10

def addA(): 1 usage
    global sequence
    for i in range(countA):
        time.sleep(timeA)
        sequence.append("A")
    print(sequence)
```

```
def addB(): 1 usage
    global sequence
    for i in range(countB):
        time.sleep(timeB)
        sequence.append("B")
    print(sequence)

if __name__ == '__main__':
    t1 = threading.Thread(target=addA)
    t2 = threading.Thread(target=addB)
    t1.start()
    t2.start()
    t1.join()
    t2.join()
```

Execution order is non-deterministic!

See code no. 4

Thread as a Subclass

```
import time
from threading import Thread

class MyThread(Thread):
    # override the constructor
    def __init__(self, x: int, y: int):
        # execute the base constructor
        super().__init__()
        # store instance variables
        self.x = x
        self.y = y
        # shared data
        self.result = 0

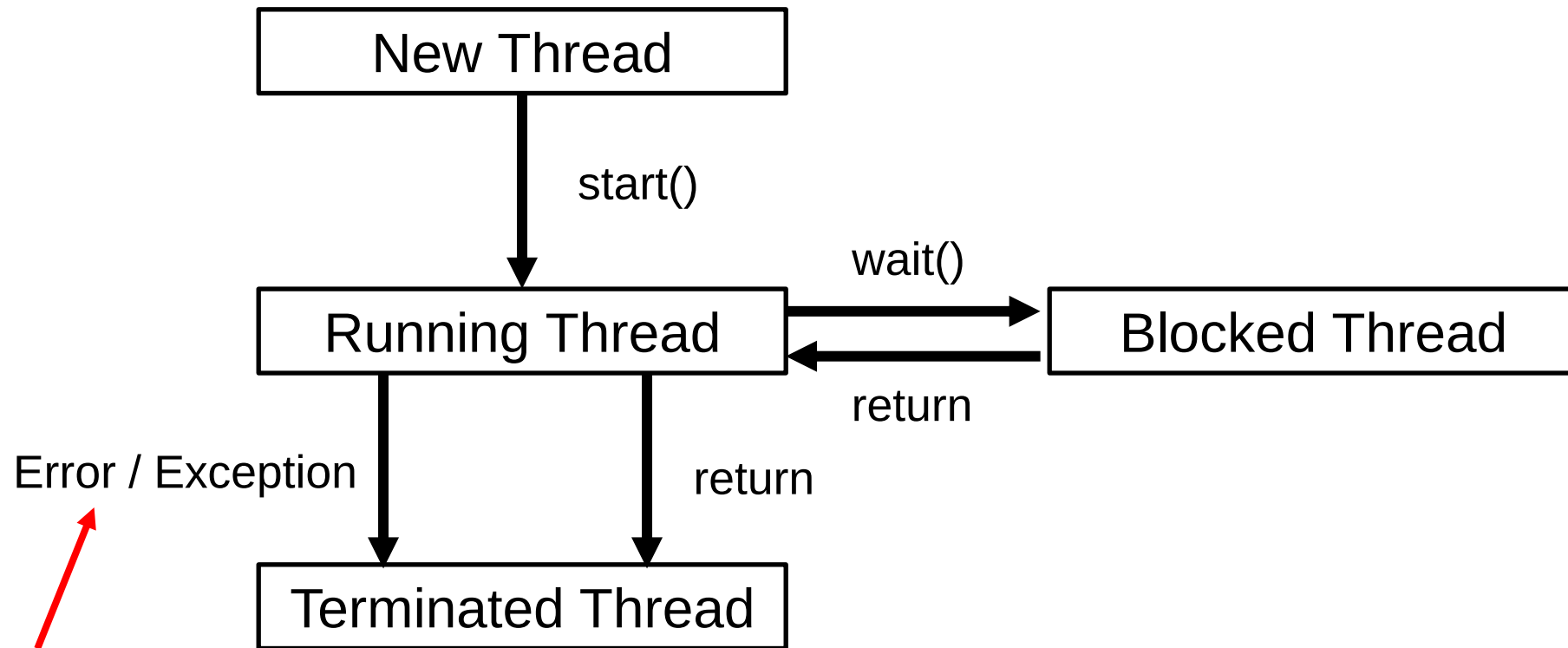
    def run(self):
        time.sleep(2)
        self.result = self.x + self.y
        # report stored value
        print(f'Child stored: {self.result}')
```

```
if __name__ == '__main__':
    thread = MyThread(x=2, y=4)
    thread.start()
    print('Waiting for the child thread to finish')
    # wait until child thread is finished
    thread.join()
    # report the thread attribute
    print(f'Parent got: {thread.result}')
```

No need for complex
communication to get
the result

See code no. 5+6

Lifecycle of Python Threads



Errors or exceptions in threads do not impact other threads or the process

What Happens Here?

Threads cannot be restarted!
They are single-use objects.

Parent starts child
Child started
Parent waits for child
Child finished
Parent ends waiting
Parent restarts child

Traceback (most recent call last):

```
File "C:\Users\e01530\PycharmProjects\PVSCChapter5\07_ThreadRestarts.py", line 18, in <module>
    thread.start()
    ~~~~~^
```

```
File "C:\Users\e01530\AppData\Local\Programs\Python\Python314\Lib\threading.py", line 988, in start
    raise RuntimeError("threads can only be started once")
```

RuntimeError: threads can only be started once

Process finished with exit code 1

See code no. 7

```
from threading import Thread
import time
```

```
def task():
    print("Child started")
    time.sleep(5)
    print("Child finished")
```

```
if __name__ == '__main__':
    thread = Thread(target=task)
    print("Parent starts child")
    thread.start()
    print("Parent waits for child")
    thread.join()
    print("Parent ends waiting")
```

```
    print("Parent restarts child")
    thread.start()
    print("Parent waits for child")
    thread.join()
    print("Parent ends waiting")
```



Relationship Between Threads and Process

- Typically, the main thread spawns another thread and waits for its return using the `join()` method
- But, if the main thread does not wait for the child, what happens to the child once the parent terminated?
- **Nothing, the child keeps running. Also the Python process keeps running. Processes are terminated when all threads exited.**

```
from threading import Thread
import time

def task(): 1 usage
    print("Child starts")
    time.sleep(10)
    print("Child finished")

if __name__ == '__main__':
    print("Parent starts")
    thread = Thread(target=task)
    thread.start()
    print("Parent finished")
```

See code no. 8



Daemon Threads

- Daemon threads (also called background threads) terminate immediately if the main thread terminates

```
from threading import Thread
import time

def task(): 1 usage
    print("Child starts")
    time.sleep(10)
    print("Child finished")

if __name__ == '__main__':
    print("Parent starts")
    thread = Thread(target=task, daemon=True)
    thread.start()
    print("Parent finished")
```

See code no. 8

Query Thread Attributes

- What will be the output here?

See code no. 9

```
from threading import Thread
import time

def printThreadDetails(thread: Thread): 4 usages
    print(f'Thread name: {thread.name}')
    print(f'Thread ID (ident): {thread.ident}')
    print(f'Thread native ID: {thread.native_id}')
    print(f'Is daemon thread: {thread.daemon}')
    print(f'Thread alive: {thread.is_alive()}')

def task(): 2 usages
    time.sleep(5)
    print('Child finished')

if __name__ == '__main__':
    thread = Thread(target=task)
    thread.start()
    printThreadDetails(thread)
    thread.join()
    printThreadDetails(thread)

    thread2 = Thread(target=task, name="Thread 2", daemon=True)
    thread2.start()
    printThreadDetails(thread2)
    thread2.join()
    printThreadDetails(thread2)
```

Identify the Main Thread

- There is no parent tracking for threads
- We can only query the main thread

See code no. 10

```
import threading
import time

def task():
    time.sleep(5)
    me = threading.current_thread()
    print(f'Child name: {me.name}')
    print(f'Child thread ident: {me.ident}')
    print(f'Child thread native id: {me.native_id}')

if __name__ == '__main__':
    thread = threading.Thread(target=task)
    thread.start()
    main_thread = threading.main_thread()
    # or: main_thread = threading.current_thread()
    print(f'Main thread name: {main_thread.name}')
    print(f'Main thread ident: {main_thread.ident}')
    print(f'Main thread native id: {main_thread.native_id}')

    thread.join()
```




Get All Active Threads

- `enumerate()` returns list of all active threads

See code no. 11

```
import threading
import time

def worker(name, delay):
    print(f'{name} started')
    time.sleep(delay)
    print(f'{name} finished')

def show_threads(label):
    print(f'--- {label} ---')
    for t in threading.enumerate():
        print(f'Name={t.name}, Alive={t.is_alive()}')

if __name__ == '__main__':
    show_threads('Before starting threads')
    t1 = threading.Thread(target=worker, args=('Worker-1', 3))
    t2 = threading.Thread(target=worker, args=('Worker-2', 5))
    t1.start()
    t2.start()
    show_threads('After starting threads')
    t1.join()
    show_threads('After joining t1')
    t2.join()
    show_threads('After joining t2')
```

Killing a Thread

- A thread cannot kill or stop another thread forcefully
- Corrupted threads might corrupt whole process
- Solution: Tell other thread it should stop
- Other threads can stop when it is safe to do so
- Easiest way: Global flag
- Better solution: Events (next chapter)

See code no. 12

```
import threading
import time

stop_flag = False

def worker():
    global stop_flag
    while not stop_flag:
        print("Working...")
        time.sleep(1)
    print("Worker exiting cleanly")

if __name__ == '__main__':
    t = threading.Thread(target=worker)
    t.start()
    time.sleep(3)
    print("Setting stop flag to True")
    stop_flag = True
    t.join()
```

ThreadPoolExecutor

```
from concurrent.futures import ThreadPoolExecutor
import time
import random

def function(i: int, t: int) -> None: 1 usage
    print(f"Thread {i} started")
    time.sleep(t)
    print(f"Thread {i} finished")

if __name__ == "__main__":
    print("PROGRAM START")
    with ThreadPoolExecutor(max_workers=4) as executor:
        for i in range(10):
            executor.submit(function, *args: i, random.randint(a: 1, b: 10))
    print("PROGRAM FINISHED")
```

See code no. 13



Thread-Local Storage

```
import threading

# Create a thread-local object
tls = threading.local()

def worker(name):
    # Each thread can set its own value
    tls.name = name
    print(f"Thread {threading.get_native_id()} has name: {tls.name}")

if __name__ == '__main__':
    # Create threads
    threads = [
        threading.Thread(target=worker, args=("Alice",)),
        threading.Thread(target=worker, args=("Bob",))
    ]

    for t in threads:
        t.start()

    for t in threads:
        t.join()
```

See code no. 14



Challenges with Threads

- Threads automatically share some resources (e.g., global variables)
- Read and write access to shared resources can lead to unpredictable behavior

```
import threading
import time

# Shared resource
counter = 0

def increment_worker(name, num_increments):
    global counter
    for _ in range(num_increments):
        current_value = counter
        # Simulate work to force context switch
        time.sleep(0.000001)
        counter = current_value + 1
    print(f'No. {name} finished')
```

```
if __name__ == '__main__':
    num_threads = 20
    n = 100
    threads = []
    for i in range(num_threads):
        t = threading.Thread(target=increment_worker, args=(f'{i+1}', n))
        threads.append(t)
        t.start()
    for t in threads:
        t.join()
    print(f'Expected counter: {num_threads * n}')
    print(f'Actual counter: {counter}')
```

See code no. 15